04. File I/O

Kyu-haeng Lee

SW Convergence Coding II

Fall 2025



DKU CI Lab

Before starting this chapter

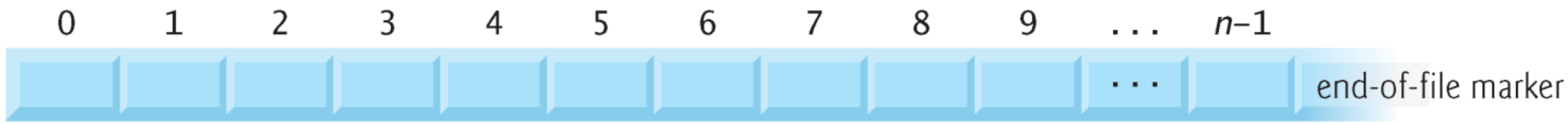
- File I/O operations are slightly different depending on which operating system you are using.

- In this chapter, MS Windows systems are assumed.

File Streams

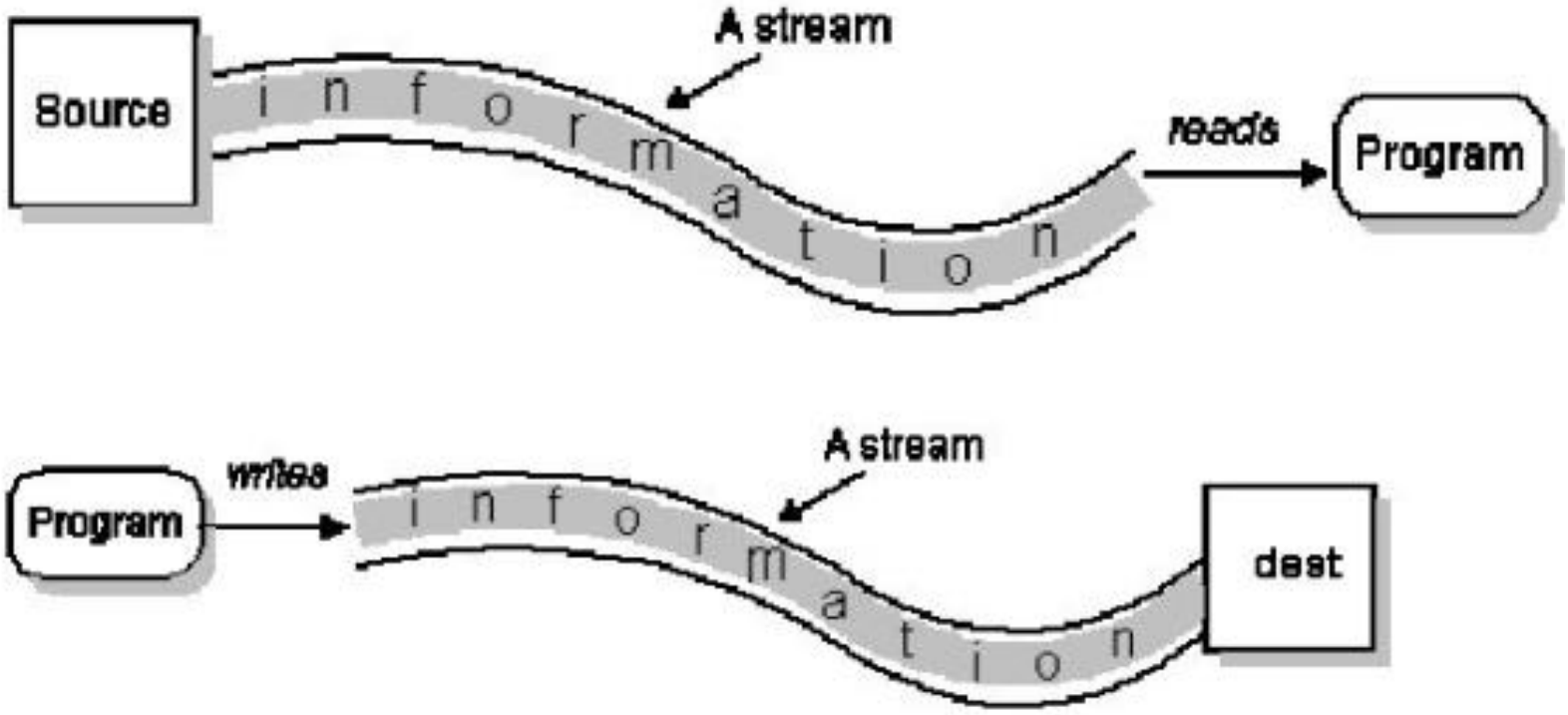
File Streams

- C views each file simply as a sequential stream of bytes
- EOF (End-Of-File)
  - Each file ends with an End-Of-File (EOF) marker.



File Streams

- When a file is opened, a stream is associated with it.
  - A stream acts as a buffer that temporarily stores data for operations (read and write).



Standard File Streams

- The following three streams are automatically opened.
  - Defined in <stdio.h>.
  - Note: devices (keyboard, screen, …) are treated as files in Linux

File Pointer	Stream	Associated Device
stdin	Standard Input	Keyboard
stdout	Standard Output	Screen
stderr	Standard Error	Screen

  - These streams are ready to use
    - We don't declare them, and we don't open or close them.

Standard File Streams

File Pointer	Stream	Associated Device
stdin	Standard Input	Keyboard
stdout	Standard Output	Screen
stderr	Standard Error	Screen

- printf and scanf
  - The role of printf is to write data to stdout which is the screen.
  - scanf read data from stdin which is the keyboard.

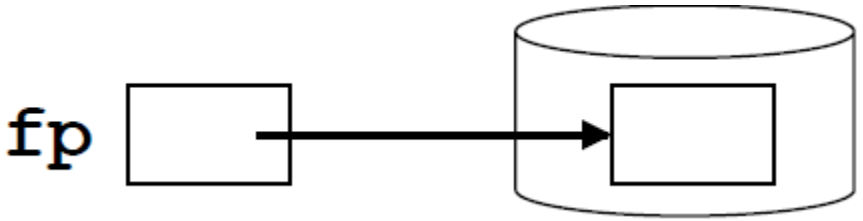
Standard File Streams

- File I/O operations do not directly access the target.
- They access its stream (i.e., buffer)!
  - printf: writes data **to the stream** connected to the screen device, not directly to the screen device.
  - scanf: reads data **from the stream** connected to the keyboard, not directly from the keyboard.

- For this property, some unexpected behavior may happen in File I/O, especially when some data remain in a buffer.

File Pointer

- Accessing a stream is done through a file pointer, which has type **FILE ***.
  - FILE *fp; // fp can point to a file stream
  - The FILE type is declared in <stdio.h>.



  - In Linux, a file pointer is mapped an nonnegative integer value.
  - Ex) stdin, stdout and stderr are represented by number 0, 1, 2.

File Pointer

- printf family
  - printf: outputs to the stdout.
    - printf("hi");
  - fprintf: outputs to a file.
    - Takes a file pointer as argument
    - fprintf(fp, "hi"); //fp is a file pointer
  - sprintf: outputs to a char array
    - Takes a char pointer as argument
    - sprintf(arr, "hi") //arr is a char pointer

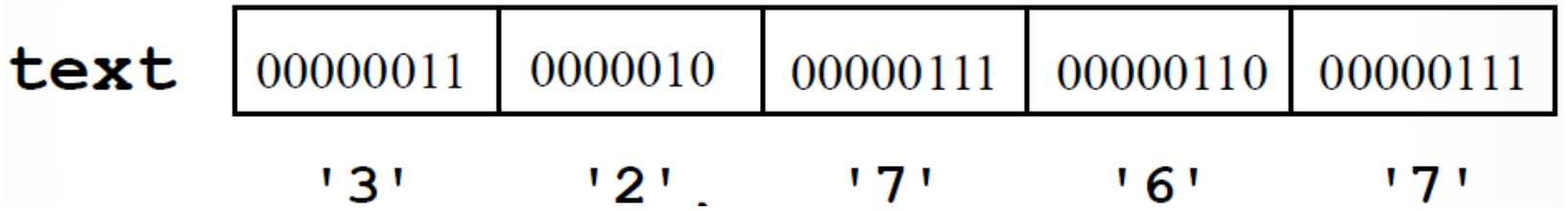
- Similarly, we have scanf, fscanf, and sscanf.

Text Files vs Binary Files

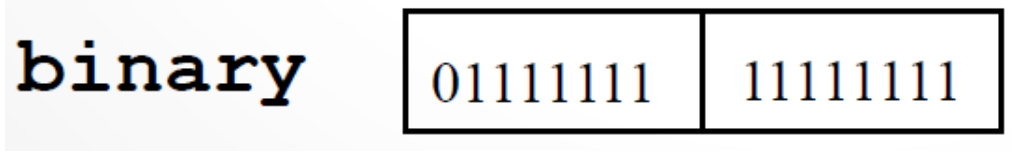
- stdio.h supports two kinds of files
  - Text file: human readable, the bytes represent characters
    - e.g., the source code for a C program.
  - Binary file: bytes don't necessarily represent characters.
    - Groups of bytes might represent other types of data, such as integers and floating-point numbers.
    - e.g., an executable C program.

Text Files vs Binary Files

- [Example] Want to write 32767 in a file
  - In a text file, you can write the numbers in text form as characters, 3, 2, 7, 6 and 7, resulting in 5 bytes of data.



  - In a binary file, the number 32767 is stored in binary, which would take only two bytes.



Differences b/w text and binary files

- Unique properties of text files (that binary files don't have)
  - 1. Divided into lines
  - 2. May contain a special EOF marker.

- In a binary file, there are no end-of-line or end-of-file markers; all bytes are treated equally.
  - When you handle files, you need to take into account whether they are text files or binary files.

- This course covers text file I/O operations.

Differences b/w text and binary files

- 1. Divided into lines
  - Each line in a text file normally ends with one or two special characters.

  - Windows: carriage-return character ('\x0d') followed by line-feed character ('\x0a')
  - UNIX and newer versions of Mac OS: line-feed character
  - Older versions of Mac OS: carriage-return character

Differences b/w text and binary files

- 2. May contain a special “end-of-file” marker.
  - In Windows, the marker is '\x1a' (Ctrl-Z)
    - Any bytes after Ctrl-Z are to be ignored.
    - Note: In some windows versions, this may not be defined.
  - Other operating systems, including UNIX, have no special end-of-file character.

File Operations

Basic functions for File I/O

- stdio.h provides a set of functions for File I/O
  - Open: A stream to the file is returned
  - Close: The stream is closed (delete all contents of the stream)
  - Read/Write: Read from/output to the stream

Opening a File

- FILE *fopen(const char * filename, const char * mode);
  - filename: the target file to open
    - The path to the file may be specified
  - mode: what operations to be performed on the file.
    - “r”: for reading, “w”: for writing
  - return: a file pointer to the stream.
    - returns a null pointer on error
    - e.g., trying to read a file that doesn’t exist.

- fp = fopen("in.dat", "r"); /* opens in.dat for reading */
  - “in.dat” must be located in the directory containing the executable file.

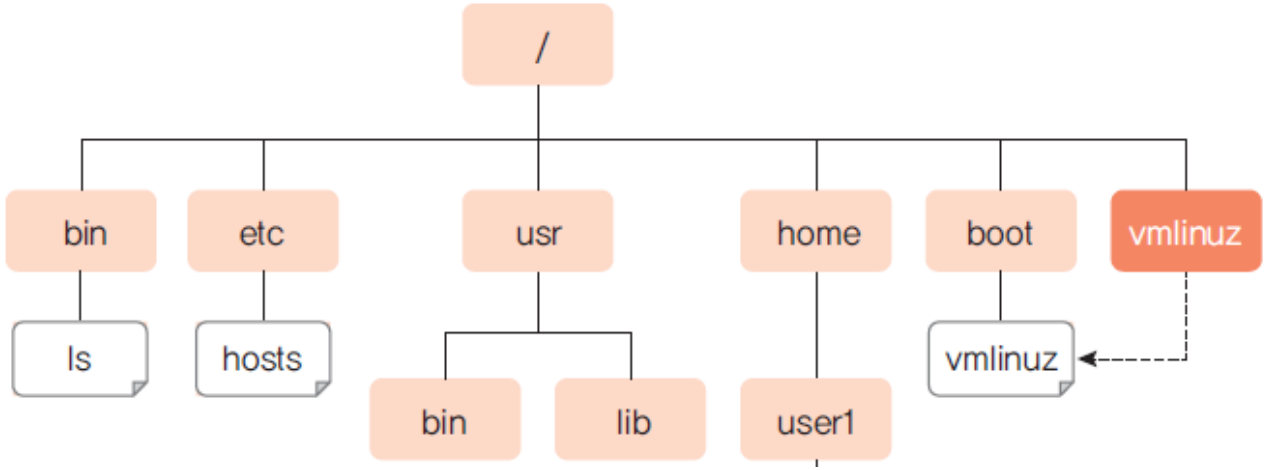
Opening a File: Mode

- Determine which mode string to pass to fopen:
  - Which operations are to be performed on the file
  - Whether the file contains text or binary data

Mode	Description
r	Open an existing file for reading.
w	Create a file for writing. If the file already exists, discard the current contents.
a	Append; open or create a file for writing at the end of the file.
r+	Open an existing file for update (reading and writing).
w+	Create a file for update. If the file already exists, discard the current contents.
a+	Append: open or create a file for update; writing is done at the end of the file.
rb	Open an existing file for reading in binary mode.
wb	Create a file for writing in binary mode. If the file already exists, discard the current contents.
ab	Append; open or create a file for writing at the end of the file in binary mode.
rb+	Open an existing file for update (reading and writing) in binary mode.
wb+	Create a file for update in binary mode. If the file already exists, discard the current contents.
ab+	Append: open or create a file for update in binary mode; writing is done at the end of the file.

Opening a File: File Path

- OSs use a hierarchical structure to manage files
- Differences b/w Linux and MS Windows FSs
  - Different directory symbols
    - Linux: / (Example: /usr/bin), Windows: \ (Example: C:\Downloads\a.txt)
  - Root
    - Linux: / (the topmost directory), Windows: drive letter (C: or D, E…)

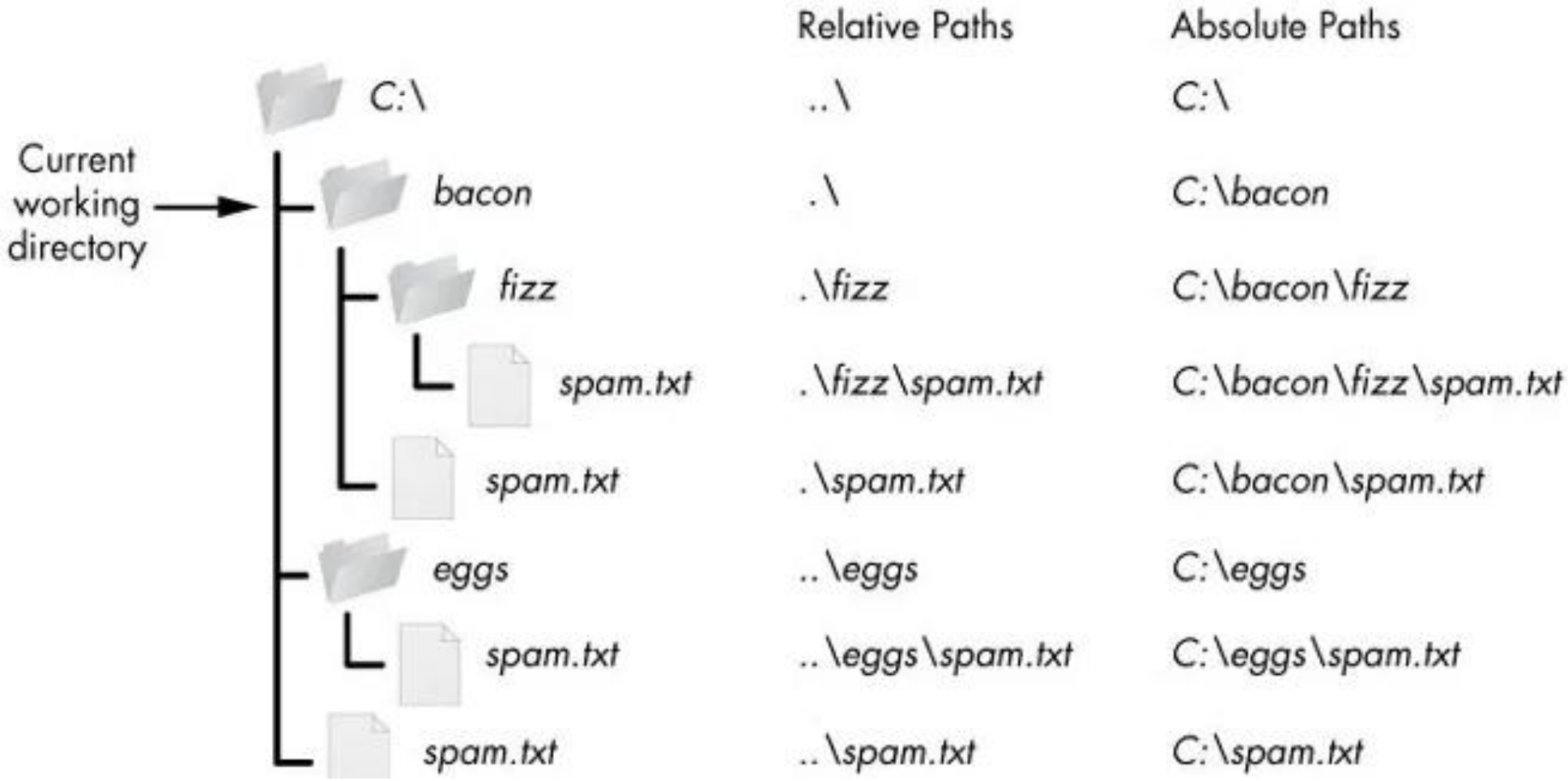


Opening a File: File Path

- Two ways to represent the file path: absolute and relative
- Absolute path: always start from the root
  - Linux: /usr/bin
  - Windows: **C:**\Downloads\a.txt
- Relative path: start from the current working directory
  - Macros for representing relative directories
    - . → current working directory
    - .. → parent directory

Opening a File: File Path

- Example



Opening a File: File Path

- In Windows, be careful when the file name in a call of fopen includes the \ character.
  - fopen("C:\project\test1.dat", "r")
  - will fail, because \t is treated as an escape character.

- One way to avoid the problem is to use \\ instead of \:
  - fopen("C:\\project\\test1.dat", "r")
- An alternative is to use the / character instead of \:
  - fopen("C:/project/test1.dat", "r")

Closing a File

- int fclose(FILE *stream)
  - Takes a file pointer obtained from fopen.
  - returns
    - zero if the file was closed successfully.
    - Otherwise, it returns the error code EOF.

Example: Open and Close

```c
#include <stdio.h>
#include <stdlib.h>
#define FILE_NAME "example.dat"

int main(void)
{
    FILE* fp;
    fp = fopen(FILE_NAME, "r");
    if (fp == NULL) {
        printf("Can't open %s₩n", FILE_NAME);
        exit(EXIT_FAILURE);
    }
    // do something
    fclose(fp);
    return 0;
}
```

Example: Open and Close

- It's not unusual to see the call of fopen combined with the declaration of fp:
  - FILE *fp = fopen(FILE_NAME, "r");

- or the test against NULL:
  - if ((fp = fopen(FILE_NAME, "r")) == NULL) …

Example: Checking Whether a File Can Be Opened

- A filename must be provided to run this program.

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char* argv[])
{
    FILE* fp;
    if (argc != 2) {
        printf("usage: canopen filename₩n");
        exit(EXIT_FAILURE);
    }
    if ((fp = fopen(argv[1], "r")) == NULL) {
        printf("%s can't be opened₩n", argv[1]);
        exit(EXIT_FAILURE);
    }
    printf("%s can be opened₩n", argv[1]);
    fclose(fp);
    return 0;
}
```

Writing and Reading

- Formatted I/O
  - Can specify the format of input/output data
  - fscanf, fprintf

- Character I/O
- Line I/O
- String I/O

Formatted I/O

- Reading – returns the number of matches or EOF
  - int fscanf(FILE *fp, "...", variableList);
  - scanf is equivalent to fscanf with stdin
    - scanf("%d%d", &i, &j); /* reads from stdin */
    - fscanf(fp, "%d%d", &i, &j); /* reads from fp */

- Writing – returns the number of chars written
  - int fprintf(FILE *fp, "...", variableList);
  - printf is equivalent to fprintf with stdout
    - printf("Total: %d\n", total); /* writes to stdout */
    - fprintf(fp, "Total: %d\n", total); /* writes to fp */

Examples of …printf Conversion Specifications

- %d conversion

Conversion Specification	*Result of Applying Conversion to 123*	*Result of Applying Conversion to –123*
%8d	•••••123	••••-123
%-8d	123•••••	-123••••
%+8d	••••+123	••••-123
% 8d	•••••123	••••-123
%08d	00000123	-0000123
%-+8d	+123••••	-123••••
%- 8d	•123••••	-123••••
%+08d	+0000123	-0000123
% 08d	•0000123	-0000123

Examples of …printf Conversion Specifications

- %s conversion

Conversion Specification	*Result of Applying Conversion to* **"bogus"**	*Result of Applying Conversion to* **"buzzword"**
%6s	•bogus	buzzword
%-6s	bogus•	buzzword
%.4s	bogu	buzz
%6.4s	••bogu	••buzz
%-6.4s	bogu••	buzz••

Examples of …printf Conversion Specifications

- The * character can be used in the specifier
  - to specify minimum field width and/or precision as argument(s)
  - The following calls produce the same output:
    - printf("%6.4d", i);
    - printf("%*.4d", 6, i);
    - printf("%6.*d", 4, i);
    - printf("%*.*d", 6, 4, i);

  - can work with a macro to specify the width or precision
    - printf("%*d", WIDTH, i);
  - the width or precision can even be computed during program execution:
    - e.g., printf("%*d", page_width / num_cols, i);

Examples of …printf Conversion Specifications

- The %p conversion: print the value of a pointer:
  - printf("%p", (void *) ptr); /* displays value of ptr */
  - The pointer is likely to be shown as an octal or hexadecimal number.

- The %n conversion: find out how many characters have been printed so far by a call of printf.
  - printf("%d%n\n", 123, &len);
    - The value of len will be 3:

The …scanf Functions

- Return Value
  - The number of data items that were successfully read.
  - EOF, if no more input characters could be read.

- Loops that test scanf's return value are common.
  - Example: A loop that reads a series of integers one by one, stopping at the first sign of trouble:

```
while (scanf("%d", &i) == 1) {
        // …
}
```

…scanf Format Strings

- The format string represents a pattern that a … scanf function attempts to match as it reads input.
- If the input doesn't match the format string, the function returns.

- Note: the input character that didn't match is "pushed back" to be read in the future. It's not removed from the buffer.

…scanf Format Strings

- "ISBN %d-%d-%ld-%d" implies
  - the letters ISBN
  - possibly some white-space characters
  - an integer
  - the - character
  - an integer (possibly preceded by white-space characters)
  - the - character
  - a long integer (possibly preceded by white-space characters)
  - the - character
  - an integer (possibly preceded by white-space characters)

scanf Examples

- Examples that combine conversion specifications, whitespace characters, and non-white-space characters:

scanf *Call*	*Input*	*Variables*
`n = scanf("%d%d", &i, &j);`	12•,•34¤	n: 1 i: 12 j: unchanged
`n = scanf("%d,%d", &i, &j);`	12•,•34¤	n: 1 i: 12 j: unchanged
`n = scanf("%d ,%d", &i, &j);`	12•,•34¤	n: 2 i: 12 j: 34
`n = scanf("%d, %d", &i, &j);`	12•,•34¤	n: 1 i: 12 j: unchanged

scanf Examples

- Examples showing the effect of assignment suppression and specifying a field width:

scanf *Call*	*Input*	*Variables*
`n = scanf("%*d%d", &i);`	12•34¤	n: 1 i: 34
`n = scanf("%*s%s", str);`	My•Fair•Lady¤	n: 1 str: "Fair"
`n = scanf("%1d%2d%3d", &i, &j, &k);`	12345¤	n: 3 i: 1 j: 23 k: 45
`n = scanf("%2d%2s%2d", &i, str, &j);`	123456¤	n: 3 i: 12 str: "34" j: 56

…scanf Conversion Specifications

- %[set] matches any sequence of characters in set (the scanset), where set can be any set of characters.

- %[^set] matches any sequence of characters not in set.

…scanf Conversion Specifications

- Examples
  - %[abc] matches any string containing only a, b, and c.
  - %[^abc] matches any string that doesn't contain a, b, or c.

***scanf* Call**	***Input***	***Variables***
`n = scanf("%[0123456789]", str);`	`123abc¤`	n: 1 str: "123"
`n = scanf("%[0123456789]", str);`	`abc123¤`	n: 0 str: unchanged
`n = scanf("%[^0123456789]", str);`	`abc123¤`	n: 1 str: "abc"

Character I/O

- Reading – returns char read or EOF
  - int fgetc(FILE *fp)
  - int getc(FILE *fp)
  - int getchar() <==> int fgetc(stdin)

- Writing – returns char written
  - int fputc(int c, FILE *fp)
  - int putc(int c, FILE *fp)
  - int putchar(int c) <==> int fputc(…, stdin)
  - int ungetc(int c, FILE *fp)

Character I/O: Reading

- ch = getchar();
  - reads a character from stdin:
- ch = fgetc(fp); ch = getc(fp);
  - read a character from an arbitrary stream:

- All three functions treat a character as an unsigned char value
  - which is then converted to int type before it's returned.
  - they never return a negative value other than EOF.

Character I/O: Reading

- A typical while loop:

```
while ((ch = getc(fp)) != EOF) {
    //…
}
```

- Always store the return value in an int variable, not a char variable.
- Testing a char variable against EOF may give the wrong result.

Character I/O: Reading

- The ungetc function “pushes back” a character read from a stream and clears the stream’s end-of-file indicator.

- Example
  - Reads a series of digits, stopping at the first non-digit:

```
while (isdigit(ch = getc(fp))) {
    //…
}
ungetc(ch, fp); /* pushes back the last character read */
```

Character I/O: Writing

- putchar(ch); /* writes ch to stdout */
  - writes one character to the stdout stream:
- fputc(ch, fp); putc(ch, fp); /* writes ch to fp */
  - write a character to an arbitrary stream:

- Example: File copy by char

```
FILE* in, * out;
// open both src and dst files as
// in and out, respectively
while ((c = fgetc(in)) != EOF) {
    fputc(c, out);
}
```

Exercise

- File copy by char
- File copy by line

Line I/O

- Reading – returns pointer to string read, NULL on end of file
  - char* fgets(char *buf, int max, FILE *fp)
  - max: the maximum number of characters to be read. (should be 1 less than the length of buf)
  - gets is equivalent to fgets(…, stdin)
- Writing – returns number of chars written
  - int fputs(char *buf, FILE *fp)

Example: File Copy by Line

```
int main() {
    char buf[BUFLEN], inFile[BUFLEN], outFile[BUFLEN];
    FILE* in, * out;
    printf("Enter source filename: ");
    fgets(inFile, BUFLEN - 1, stdin);
    inFile[strlen(inFile) - 1] = '₩0' ;
    // get outFile as well from the user
    in = fopen(inFile, "r");
    out = fopen(outFile, "w");
    if ((in == NULL) || (out == NULL)) {
        printf("*** File open error₩n");
        return;
    }
    /* NULL returned at EOF */
    while (fgets(buf, BUFLEN - 1, in) != NULL) {
        fputs(buf, out);
    }
    fclose(in); fclose(out);
    return 0;
}
```

String I/O

- Read and write data using a string as if it were a stream.
- sprintf
  - writes output into a character array (pointed to by its first argument) instead of a stream.
  - adds a null character at the end of the string.
  - returns the number of characters stored (not counting the null character).

  - sprintf(date, "%d/%d/%d", 9, 20, 2010);
    - writes "9/20/2010" into date

String I/O

- sscanf
  - reads characters from a string.
  - returns the number of data items successfully read and stored.
  - returns EOF if it reaches the end of the string (marked by a null character) before finding the first item.

- Example
  - fgets(str, sizeof(str), stdin); /* reads a line of input */
  - sscanf(str, "%d%d", &i, &j); /* extracts two integers */
  - uses fgets to obtain a line of input, then passes the line to sscanf for further processing:

Exercise

- Parsing

String I/O

- One advantage of using sscanf is that we can examine an input line as many times as needed. This makes it easier to recognize alternate input forms and to recover from errors.

```
if (sscanf(str, "%d /%d /%d", &month, &day, &year) == 3)
    printf("Month: %d, day: %d, year: %d₩n", month, day, year);
else if (sscanf(str, "%d -%d -%d", &month, &day, &year) == 3)
    printf("Month: %d, day: %d, year: %d₩n", month, day, year);
else
    printf("Date not in the proper form₩n");
```

File I/O on Memory Blocks: Binary Files

- Binary files do not have a line-oriented structure.
- They consist of blocks of objects, and the format of these objects are application as well as machine dependent.

- When opening a binary file, always append a 'b' to the second argument, i.e.
  - fopen("test.exe", "wb");
  - fopen("test.exe", "rb");

File I/O on Memory Blocks: Binary Files

- The fread and fwrite functions allow a program to read and write large blocks of data in a single step.
  - size_t fread(void *buf, size_t s, size_t ne, FILE *in);
    - reads up to ne elements, each of size s in bytes and stores them in buf
  - size_t fwrite(const void *buf, size_t s, sizt_t ne, FILE *out);
    - similarly, writes from buf up to ne blocks, each of size s

  - Number of elements (not bytes!) successfully read/written is returned

Exercise

- Read a text and write it as a binary.
- Read a binary.

File I/O on Memory Blocks: Binary Files

- Example: reads the contents of a file into the array a:
  - n = fread(a, sizeof(a[0]), sizeof(a) / sizeof(a[0]), fp);

- Example: writes a structure variable s to a file:
  - fwrite(&s, sizeof(s), 1, fp);
  - fwrite is convenient for a program that needs to store data in a file before terminating.
  - Later, the program (or another program) can use fread to read the data back into memory.

File Positioning

- Each file has an associated file position
- When a file is opened, the file position is set either at the beginning or the end
  - SEEK_SET – beginning of file
  - SEEK_CUR – current file position
  - SEEK_END – end of file

  - int fseek(FILE *fp, long offset, int whence)
  - void rewind(FILE *fp)
  - rewind(fp) ←→ fseek(fp, 0L, SEEK_SET)

File Positioning

- long ftell(FILE *fp);
  - returns the current file position, or –1L.
- fseek(fp, -sizeof(double), SEEK_CUR);
  - set the file position
- ftell's return value is often used by fseek:

```
long file_pos;
//…
file_pos = ftell(fp);
/* saves current position */
//…
fseek(fp, file_pos, SEEK_SET);
/* returns to old position */
```

Exercise

- ftell, fseek,
- fflush

File Positioning

- fgetpos(fp, &file_pos)
  - stores the file position associated with fp in the file_pos variable.
- fsetpos(fp, &file_pos)
  - sets the file position for fp to be the value stored in file_pos.

```
fpos_t file_pos;
//…
fgetpos(fp, &file_pos);
/* saves current position */
//…
fsetpos(fp, &file_pos);
/* returns to old position */
```

File Buffering

- fflush function
  - A call that flushes the buffer for the file associated with fp
  - fflush(fp); /* flushes buffer for fp */

  - A call that flushes all output streams:
  - fflush(NULL); /* flushes all buffers */
  - Note: it's for stdout, not for stdin.

  - fflush returns zero if it's successful and EOF if an error occurs.

References

- Chapter 11. C File Processing, C: How to Program, Paul Deitel et al.
- Chapter 22. Input/Output, C Programming, K. N. King.